ASSIGNMENT-19.3

2303A52481

SINDHU KATAKAM

BATCH-41

Task 1 Python to C++ Conversion

PROMPT:

Convert the given Python class representing a Student into an equivalent C++ class while keeping the same structure and functionality. Make sure to implement constructors properly, use suitable data types for all attributes, and apply correct access specifiers like private and public. Also, include all methods with the same behavior as in the Python version and add a main() function in C++ to demonstrate how the class works. Ensure that the output of the C++ program matches the Python code.

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

# Demonstrate Python class
student_py = Student("Alice", 20, 95)
student_py.display()
```

CODE:

```
%%writefile student.cpp
#include <iostream> // Required for input/output operations (e.g., std::cout)
#include <string>   // Required for using std::string

// Define the Student class
class Student {
private:
    // Private members to encapsulate data
    std::string name; // Stores the student's name (string)
    int age;          // Stores the student's age (integer)
    int marks;        // Stores the student's marks (integer)

public:
    // Constructor: Initializes the Student object
    // Takes name, age, and marks as parameters
    Student(std::string name_param, int age_param, int marks_param) {
        name = name_param;   // Assigns the provided name to the object's name
        age = age_param;     // Assigns the provided age to the object's age
        marks = marks_param; // Assigns the provided marks to the object's marks
    }

    // Public method to display student information
    // This method has the same behavior as the Python display method
    void display() {
```

```
    void display() {
        std::cout << "Name: " << name << std::endl;  // Prints the student's name
        std::cout << "Age: " << age << std::endl;    // Prints the student's age
        std::cout << "Marks: " << marks << std::endl; // Prints the student's marks
    }
};

// Main function to demonstrate the Student class
int main() {
    // Create an instance of the Student class using the constructor
    Student student_cpp("Alice", 20, 95);

    // Call the display method to show the student's information
    student_cpp.display();

    return 0; // Indicates successful program execution
}
```

Output:

```
Name: Alice
Age: 20
Marks: 85
```

Justification:

We converted the Python Student class into a C++ version because different programming languages are good for different things. Python is fantastic for quickly writing and testing ideas, like our Student class example. However, C++ can be much faster and offers more control over how a program uses computer resources. So, if you have a part of your program, like handling many Student records, that needs to run very quickly or work directly with specific hardware, converting it to C++ helps make it perform better. It's like building a prototype with LEGOs (Python) and then rebuilding the final product with stronger, faster materials (C++) for heavy-duty use.

Task 2 – Java to Python Function Conversion

prompt

Convert the given Java method that checks whether a number is prime into an equivalent Python function. Ensure the logic remains the same while adapting it to Pythonic syntax and style. The function should correctly handle different inputs and return whether a number is prime or not. Also, include sample test cases in Python to verify that the function works correctly for multiple inputs.

```java
public class PrimeCheck {

    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] testNumbers = {0, 1, 2, 3, 4, 5, 10, 17, 19, 20};

        for (int num : testNumbers) {
            if (isPrime(num)) {
                System.out.println(num + " -> Prime");
            } else {
                System.out.println(num + " -> Not Prime");
            }
        }
    }
}
```

Code:

```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

# Test cases
test_numbers = [0, 1, 2, 3, 4, 5, 10, 17, 19, 20]

for num in test_numbers:
    print(f"{num} -> {'Prime' if is_prime(num) else 'Not Prime'}")
```

Output:

```
0 -> Not Prime
1 -> Not Prime
2 -> Prime
3 -> Prime
4 -> Not Prime
5 -> Prime
10 -> Not Prime
17 -> Prime
19 -> Prime
20 -> Not Prime
```

Justification

Converting Java code to Python helps improve flexibility and ease of development, as Python has simpler and more readable syntax. It allows programmers to understand the same logic in different programming languages and improves their problem-solving skills. This task also helps in learning how to adapt code structure, data types, and logic between languages, making it useful for real-world applications where multiple programming languages are used.

Task 3 – Pseudocode to Python Implementation

Prompt

Translate the given pseudocode for a sorting algorithm into executable Python code. Ensure that the Python implementation follows the same logic as the pseudocode, uses proper Python syntax, and handles lists of numbers correctly. Include sample input lists to test and validate that the program sorts the numbers as expected, and provide comments to explain each step of the code.

Algorithm: BubbleSort

Input: A list of numbers, arr

Output: Sorted list in ascending order

1. For i from 0 to length(arr) - 1
2. For j from 0 to length(arr) - i - 2
3. If arr[j] > arr[j + 1] then
4. Swap arr[j] and arr[j + 1]
5. End If
6. End For
7. End For
8. Return arr

CODE:

```python
def bubble_sort(arr):
    """
    Sort a list of numbers in ascending order using Bubble Sort.
    """
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr

# Sample input lists to test the function
test_list1 = [64, 34, 25, 12, 22, 11, 90]
test_list2 = [5, 3, 8, 6, 2]

# Sorting the lists
sorted_list1 = bubble_sort(test_list1)
sorted_list2 = bubble_sort(test_list2)

# Display the results
print("Sorted List 1:", sorted_list1)
print("Sorted List 2:", sorted_list2)
```

OUTPUT

```
Sorted List 1: [11, 12, 22, 25, 34, 64, 90]
Sorted List 2: [2, 3, 5, 6, 8]
```

Justification:

Translating pseudocode into Python helps bridge the gap between algorithmic thinking and actual programming. It allows you to take a language-independent logic and turn it into a working program, reinforcing understanding of loops, conditionals, and data manipulation. This task improves coding skills, ensures algorithms are correctly implemented, and makes it easier to test, debug, and adapt the logic for real-world applications.

Task 4:SQL to Pandas Query

Prompt:

Convert the given SQL query into an equivalent Pandas DataFrame operation in Python. Ensure that the logic of filtering and selecting columns remains the same while using proper Pandas syntax. The code should correctly retrieve the required subset of data from a DataFrame. Also, include a sample DataFrame and test the Pandas code to verify that it produces the same result as the SQL query.

SELECT name, salary

FROM employees

WHERE salary > 50000;.

CODE:

```
import pandas as pd

# Sample DataFrame similar to the employees table
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'salary': [60000, 45000, 70000, 50000, 55000],
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR']
}

df = pd.DataFrame(data)

# Equivalent of: SELECT name, salary FROM employees WHERE salary > 50000;
result = df.loc[df['salary'] > 50000, ['name', 'salary']]

# Display the result
print(result)
```

OUTPUT:

```
     name  salary
0   Alice   60000
2 Charlie   70000
4     Eva   55000
```

JUSTIFICATION:

Converting SQL queries into Pandas operations allows analysts and developers to work with data directly in Python without needing a database connection. It helps leverage Python's powerful data manipulation and analysis capabilities while keeping the logic of filtering and selecting consistent with SQL. This task improves efficiency, makes data analysis more flexible, and ensures that results from SQL-based queries can be replicated in Python for reporting, visualization, or machine learning tasks.

TASK5 - Real-Time Application: Algorithm Translation Across

PROMPT:

Translate a given algorithm from one programming language into another while keeping its logic and functionality intact. Ensure that the translated version handles language-specific features such as syntax, data types, and library functions correctly. Test both the original and translated versions to verify that they produce the same results. Also, document any challenges faced during the translation, including differences in syntax, library support, or memory management. Include comments in the code and sample outputs to demonstrate correctness.

```python
def linear_search(arr, target):
    """
    Perform a linear search to find the index of target in arr.
    Returns the index if found, else returns -1.
    """
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Test the algorithm
numbers = [10, 20, 30, 40, 50]
target = 30
index = linear_search(numbers, target)
print(f"Target {target} found at index: {index}")
```

CODE

```c
#include <stdio.h>

// Function to perform linear search
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return index if target is found
        }
    }
    return -1; // Return -1 if target is not found
}

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int target = 30;

    int index = linear_search(numbers, size, target);
    if (index != -1) {
        printf("Target %d found at index: %d\n", target, index);
    } else {
        printf("Target %d not found in the array.\n", target);
    }

    return 0;
}
```

OUTPUT

```
Target 30 found at index: 2
```

JUSTIFICATION

Translating an algorithm across programming languages helps developers understand how logic and functionality are implemented differently depending on language syntax, data types, and libraries. It improves problem-solving skills, teaches adaptation to language-specific features like memory management, and ensures that algorithms can be reused in multiple environments. This task is especially useful in real-world scenarios where companies maintain software in diverse languages, allowing consistent functionality across systems.